

Harness Teardown — Gold Example

Hand out after the debrief · the seven planted flaws + the order to fix them

The one idea to leave with: a harness is a **control surface**. Almost none of these failures are about the model's quality — they're about the plumbing that turns “the model said something” into “money left the bank.” The model is the part everyone validates; the harness is the part that moves the money.

```

1  # Meridian Agent Runtime v0.1 — shared harness sketch
2  agent = Agent(model="meridian-llm", system_prompt=SYSTEM)
3  tools = load_all_tools()          # every registered tool
4  context = [SYSTEM, user_request]
5
6  while True:                        # main loop
7      response = agent.run(context, tools)
8      if response.tool_calls:
9          for call in response.tool_calls:
10             result = call.execute()    # runs immediately
11             context.append(result.raw) # raw output back into context
12      else:
13          context.append(response.text)
14      # ... loop continues ...
15
16  # registered tools: read_file, web_fetch, send_email,
17  # post_ledger_entry, execute_payment, run_sql, shell
18
19  def execute(call):
20      try:
21          return tool.invoke(call.args)
22      except:
23          pass                    # on error, keep going
24
25  # SYSTEM prompt contains API_KEY and DB_PASSWORD
26  # logging: none
27  # result returned to user only when the loop ends

```

The seven flaws

Line 6 · No termination contract

HIGH

`while True` has no max-iteration cap and never breaks when the model stops calling tools (lines 12–13 append text but never return) — the loop can run forever, burning cost and repeating actions.

Smallest fix: Cap iterations and break when there are no tool calls (or on an explicit `[DONE]` sentinel).

Bites hardest: Every pod — runaway cost and repeated side effects.

Line 3 · Unscoped tool access

HIGH

`load_all_tools()` hands the agent every registered tool — including `execute_payment`, `post_ledger_entry`, `run_sql`, and `shell` — no matter the task.

Smallest fix: Pass a per-use-case allowlist; default deny.

Bites hardest: SettleNet (posting), Navigator (account actions).

Line 10 · No checkpoint before irreversible actions

HIGH

`call.execute()` fires immediately, so money-movement and send actions commit with no human gate and no dry-run.

Smallest fix: Route irreversible tools through an approval step before execution.

Bites hardest: Pod 03 Navigator, Pod 04 SettleNet.

Line 11 · Unvalidated tool output into context

HIGH

`result.raw` — including `web_fetch` results and retrieved documents — is appended verbatim, so attacker-controlled text becomes model instructions (prompt injection / cross-agent injection).

Smallest fix: Treat tool and retrieved output as untrusted; fence or sanitize it, don't append raw.

Bites hardest: Navigator (chat field), SettleNet (vendor agent).

Line 20–23 · Swallowed errors

MED

A bare `except: pass` means a failed tool call is ignored and the agent proceeds on bad or partial state.

Smallest fix: Handle errors explicitly — halt or surface them; never `pass`.

Bites hardest: All — silent corruption of the run.

Line 25 · Secrets in the prompt

HIGH

`API_KEY` and `DB_PASSWORD` live in the system prompt, so any injection or leaked transcript exposes them.

Smallest fix: Pull secrets from a runtime secrets manager; keep them out of the prompt and context.

Bites hardest: All — credential exposure.

Line 26 · No logging / observability

MED

Nothing records the trajectory, so you can't audit, debug, or produce an MRM artifact after the fact.

Smallest fix: Log every step — the full transcript / trajectory — to durable storage.

Bites hardest: Pod 05 Second Line (MRM), all.

WHICH TO FIX FIRST

Cap the blast radius before anything else: **scope the tools (line 3)** and **gate the irreversible actions (line 10)** — together they are the path from a bad token to an executed payment. Then close the **injection path (line 11)**.

Termination (line 6) and **swallowed errors (lines 20–23)** stop runaway cost and silent corruption. **Secrets (line 25)** and **logging (line 26)** are exposure and blindness — fix them, but they don't move money on their own.

Illustrative for the exercise only. The sketch is intentionally broken and simplified; a real runtime has more moving parts. Nothing here is a reference architecture.